

Applications of Transformer

(BERT, GPT, BART, T5, SpanBERT)

Md Shad Akhtar
shad.akhtar@iiitd.ac.in



INDRAPRASTHA INSTITUTE of
INFORMATION TECHNOLOGY DELHI

Bidirectional Encoder Representation from Transformer (BERT)

[Devlin et al., 2018] <https://www.aclweb.org/anthology/N19-1423.pdf>

BERT

- Bi-directional Training of Transformer for Language Modelling
 - Its an encoder-only model.
- Classically, LM is a unidirectional approach, hence,
 - BERT modified the strategy of LM to Masked Language Modelling (MLM)
- Additionally, it also incorporates Next-Sentence Prediction (NSP)

Unsupervised (Pre-)Training

- Almost all of the pre-trained models (e.g., w2v, glv, BERT, GPT, etc.) leverages unsupervised training.
- Unsupervised
 - Training data is obtained without human intervention
- Training
 - Once the training data is obtained, we train our network in a classical supervised fashion, i.e., computing loss and propagating its gradient back to the network.

Masked Language Modelling

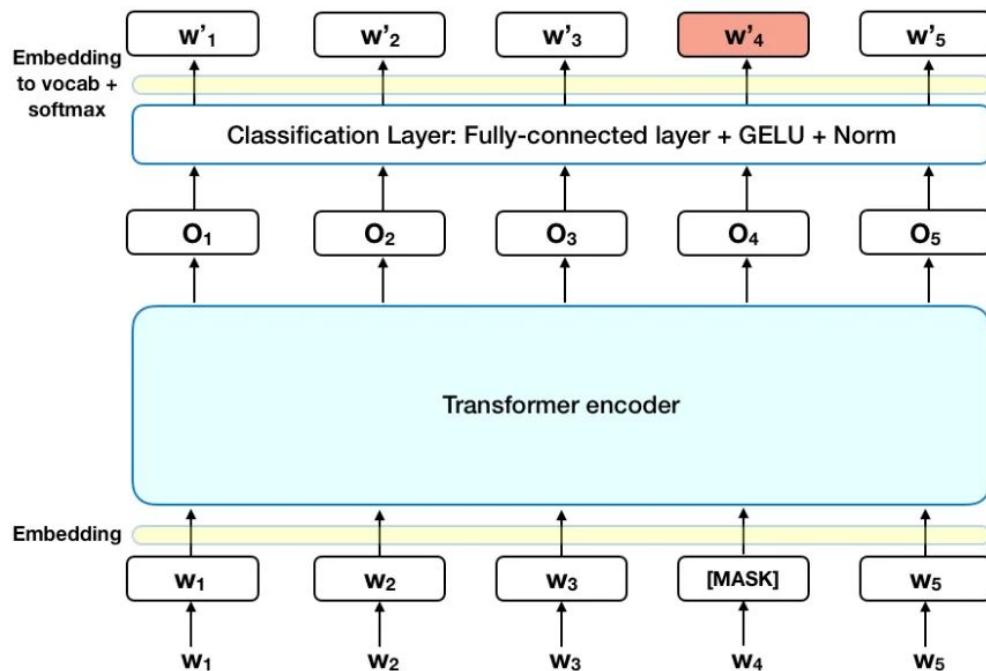
- At first, 15% of the tokens in each sequence are replaced with mask tokens
- BERT attempts to Predicts the masked tokens(s) to complete the sentence.
 - BERT loss function ignores the prediction of the non-masked words

Raw data from any corpus:

- John ate an apple. He was hungry.

Training data for MLM ($x \rightarrow y$):

- [John <masked> an apple .] → [John ate an apple .]
- [<masked> ate an apple .] → [John ate an apple .]
- [John ate an <masked> .] → [John ate an apple .]
- [<masked> was hungry .] → [He was hungry .]
- ..
- ..



Next Sentence Prediction

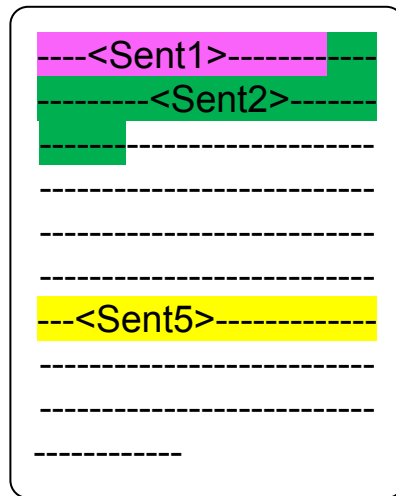
- Given a sentence pair, it predicts if the second sentence is a logical continuation of the first sentence or not.
- During training, a 50-50 strategy was adopted for the sentence and non-sentence pairs.

Training data for NSP ($x \rightarrow y$):

[ -----<Sent1>-----  -----<Sent2>-----] $\rightarrow$ True

[ -----<Sent5>-----  -----<Sent1>-----] $\rightarrow$ False

Raw data from any corpus



BERT training instances

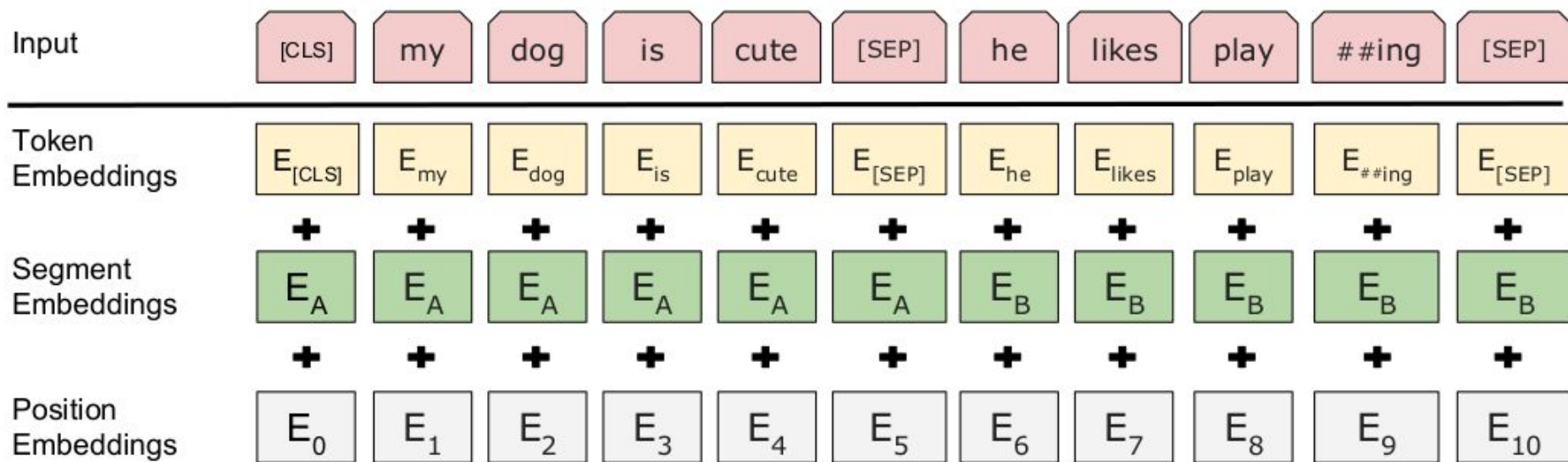
Input = [CLS] the man went to [MASK] store [SEP] he bought a gallon [MASK] milk [SEP]
Label = IsNextSentence!

Input = [CLS] the man [MASK] to the store [SEP] penguin [MASK] flight ##less birds [SEP]
Label = NotNextSentence!

[CLS] and [SEP] are two special tokens, where,

- [SEP] acts as a segregator of two or more input units;
- [CLS] denotes the overall input and is always the first token.

BERT input representation



BERT tokenizer

- Uses Word-Piece tokenizer [Schuster et al., 2012]
 - Based on BPE (Discussed in the Text Processing lecture)
 - Uses word boundary token at the beginning of the word instead of at the end.
 - Rather than merging two most-frequent pairs, it merges if the pair maximizes the likelihood of the training data once added to the vocabulary”.
- Breaks down a token into known sub-tokens.
 - Useful for handling rare and unseen tokens.

```
from transformers import BertTokenizerFast
tokenizer = BertTokenizerFast.from_pretrained("google-bert/bert-base-uncased")

def BERTTokenizedOutput(sample):
    encoding = tokenizer.encode(sample)
    print(tokenizer.convert_ids_to_tokens(encoding))

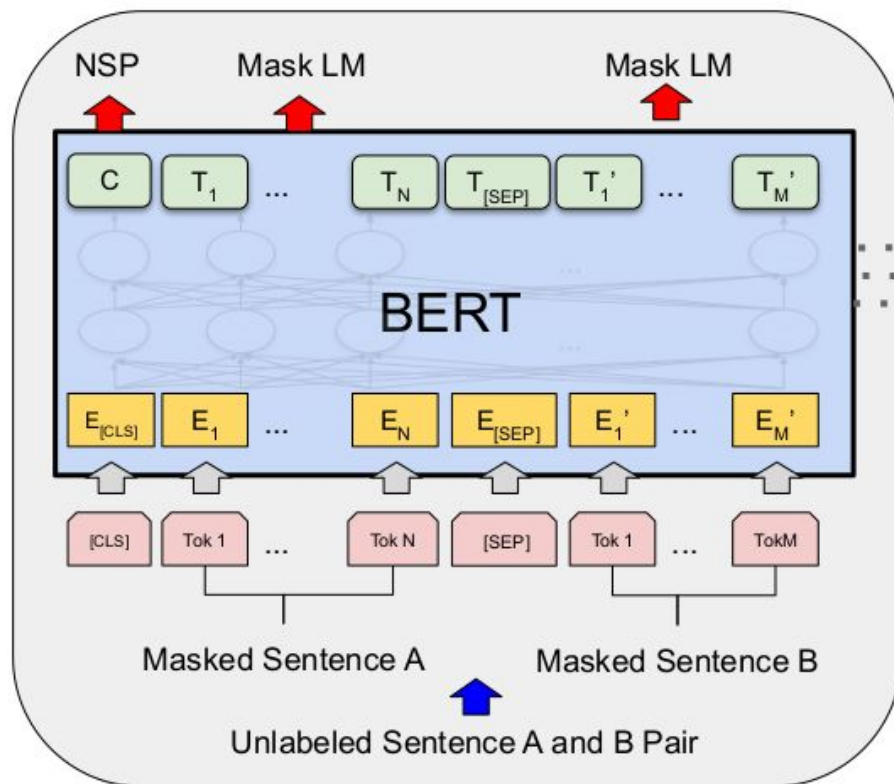
BERTTokenizedOutput('penguin are flightless birds') # ['[CLS]', 'penguin', 'are', 'flight', '##less', 'birds', '[SEP]']

BERTTokenizedOutput('He likes playing cricket.') # ['[CLS]', 'he', 'likes', 'playing', 'cricket', '.', '[SEP]']

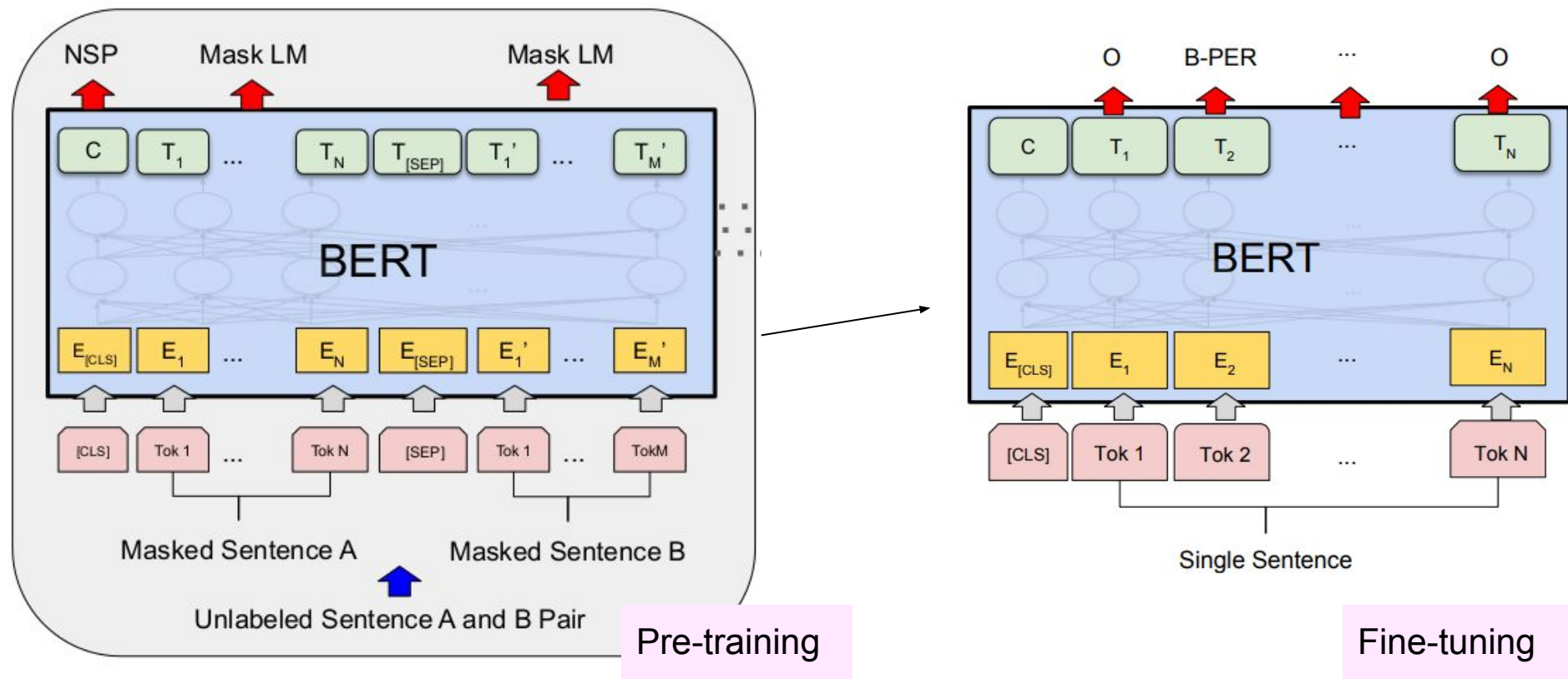
BERTTokenizedOutput('He likes playing crickat') # ['[CLS]', 'he', 'likes', 'playing', 'cr', '##ick', '##at', '[SEP]']
```

BERT as a foundational model

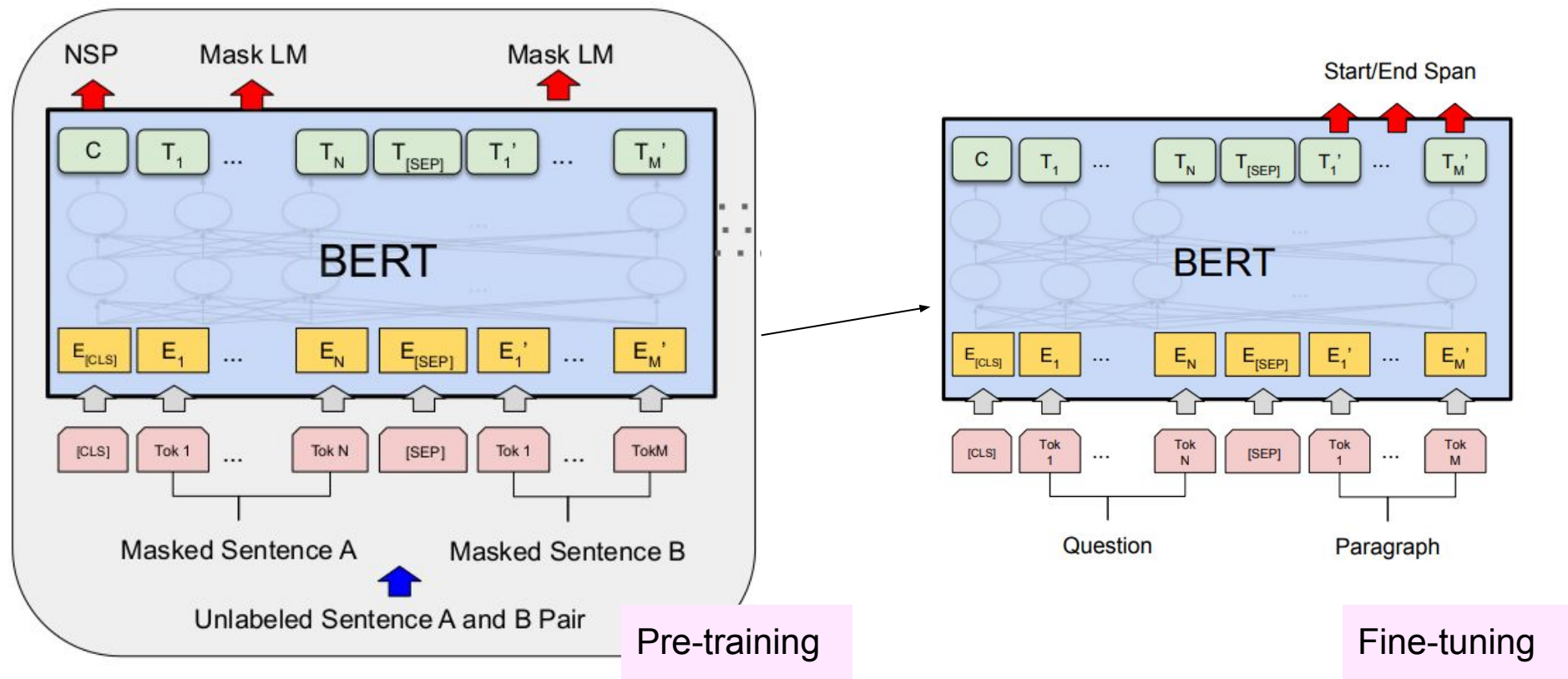
BERT training



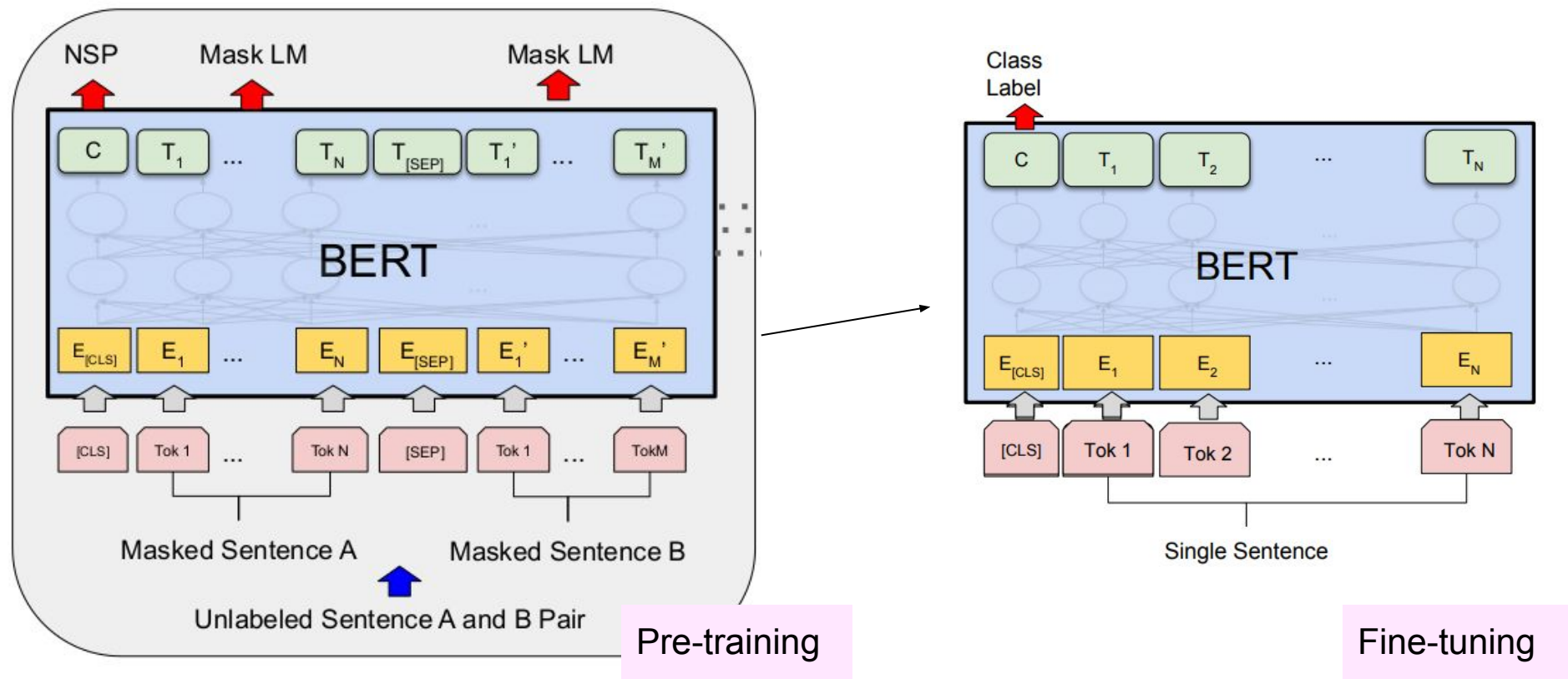
BERT fine-tuning: Sequence-labelling (NER)



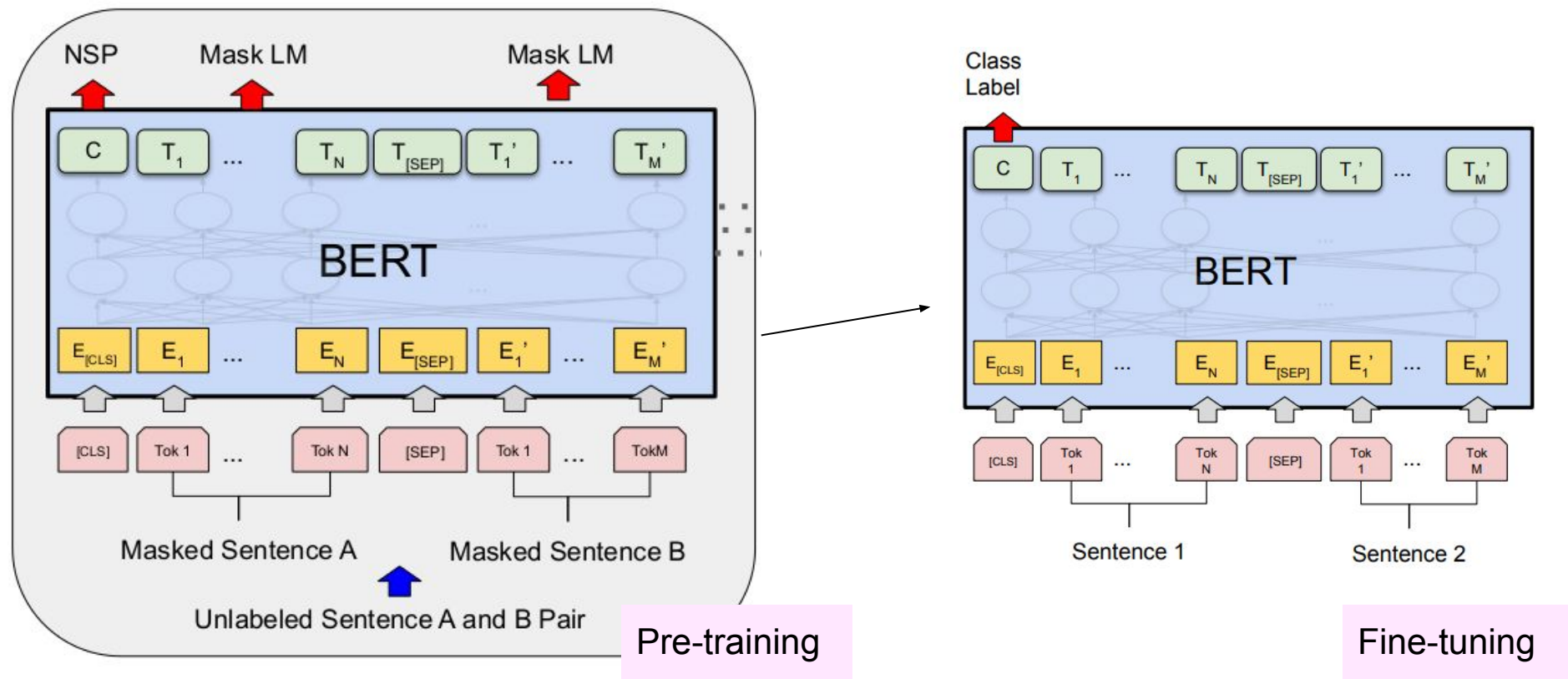
BERT fine-tuning: Sequence-labelling (Answer-extraction)



BERT fine-tuning: Single-sentence Classification (Sentiment)



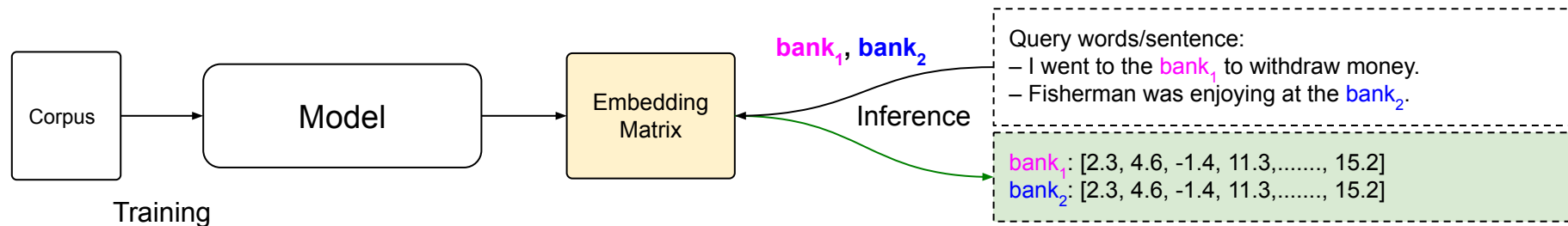
BERT fine-tuning: Sentence-pair Classification (MNLI, TE)



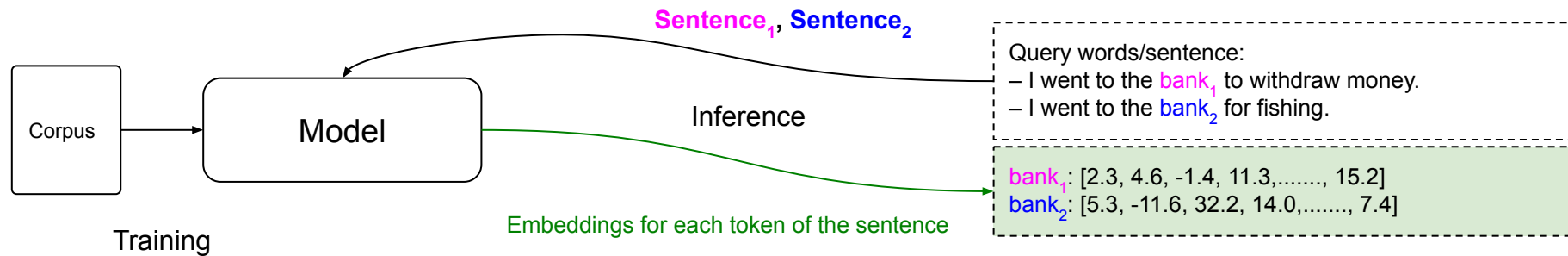
BERT for obtaining word representations

BERT embeddings

- It belongs to the class of contextual word embeddings
 - Word2vec, GloVe, Fasttext are the examples of non-contextual embeddings.

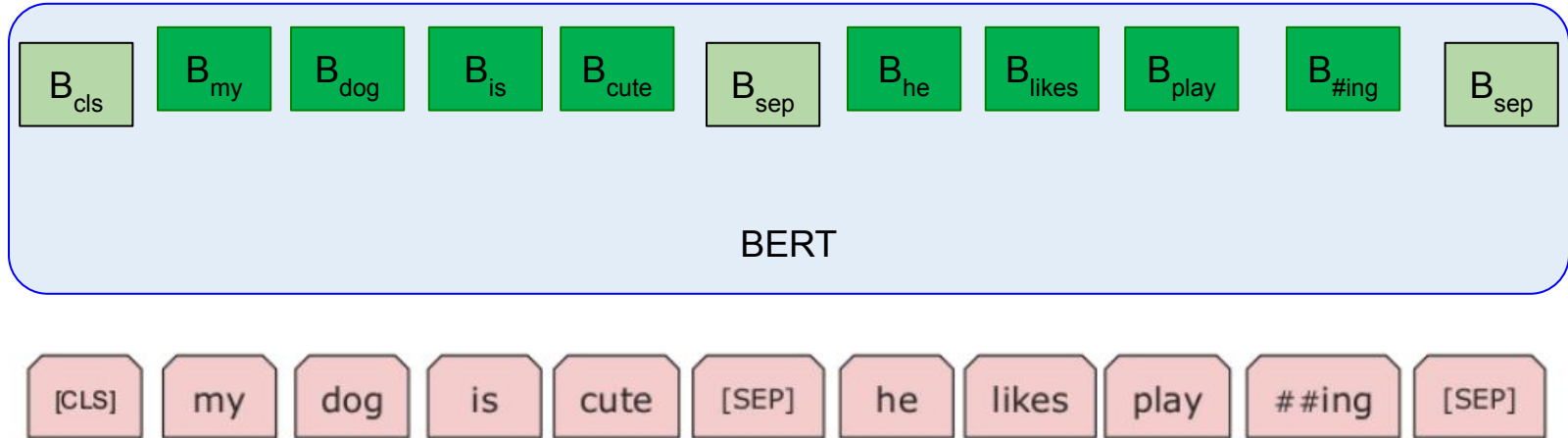


Non-contextual WE



Contextual WE

BERT embeddings



Generative Pretraining Transformer (GPT)

[Radford et al., 2018]

https://s3-us-west-2.amazonaws.com/openai-assets/research-covers/language-unsupervised/language_understanding_paper.pdf

GPT

- Objective is to generate text (i.e., LM) instead of MLM of BERT
- It's a decoder only model and auto-regressive in nature.
 - Does not know/utilize the future text.
- Optimizes two objectives:

$$h_0 = UW_e + W_p$$

$$h_l = \text{transformer_block}(h_{l-1}) \forall i \in [1, n]$$

$$P(u) = \text{softmax}(h_n W_e^T)$$

$$L_1(\mathcal{U}) = \sum_i \log P(u_i | u_{i-k}, \dots, u_{i-1}; \Theta)$$

Unsupervised pre-training of Standard LM objective

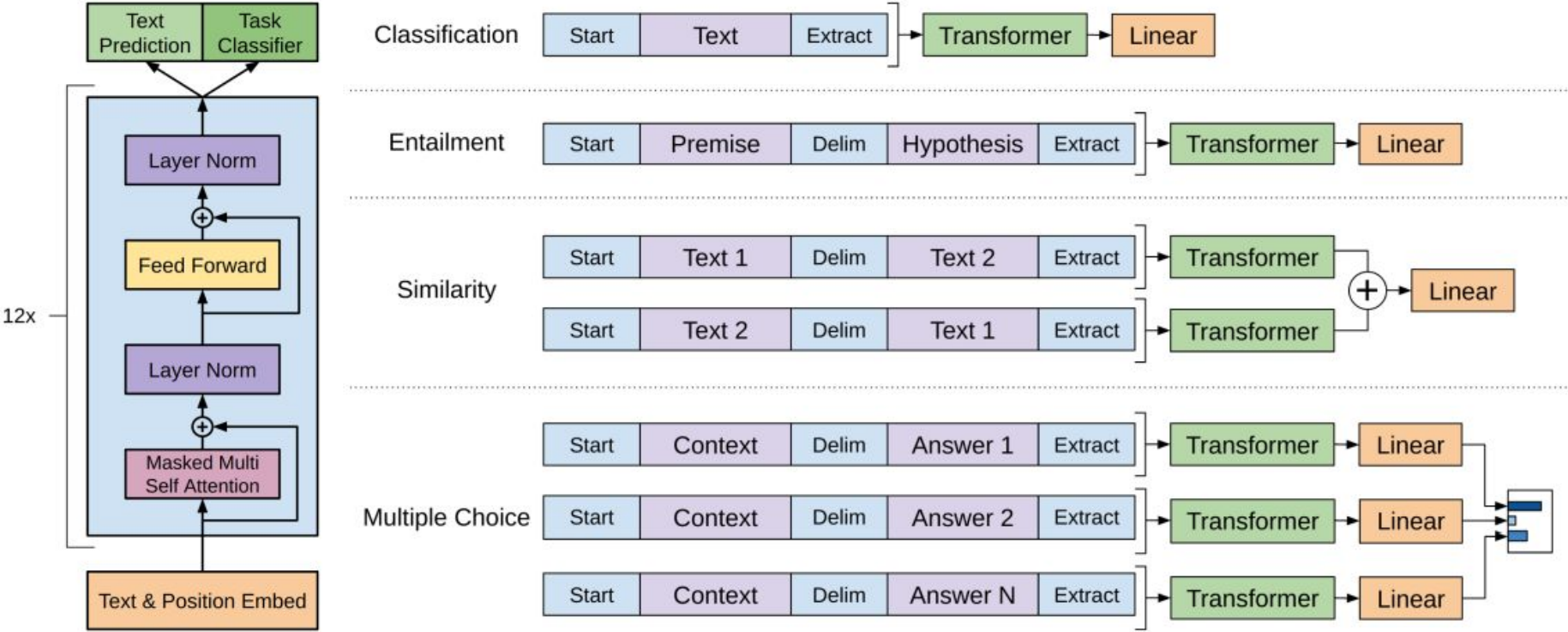
$$P(y|x^1, \dots, x^m) = \text{softmax}(h_l^m W_y).$$

$$L_2(\mathcal{C}) = \sum_{(x,y)} \log P(y|x^1, \dots, x^m).$$

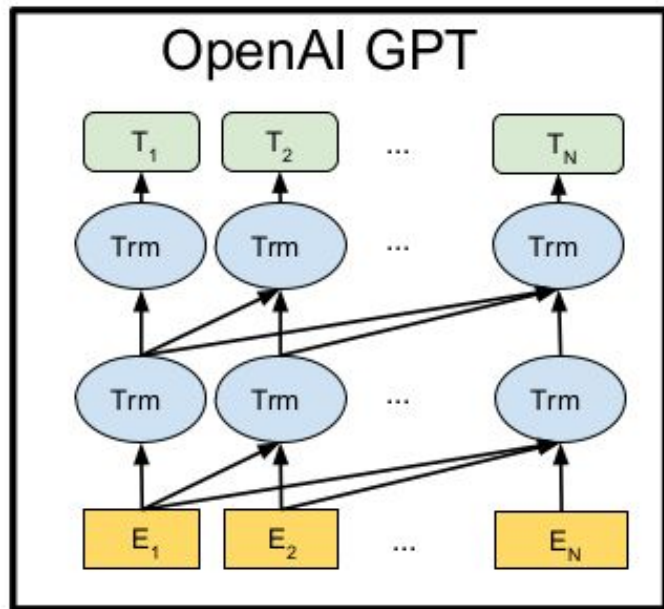
$$L_3(\mathcal{C}) = L_2(\mathcal{C}) + \lambda * L_1(\mathcal{C})$$

Supervised fine-tuning on classification tasks

GPT

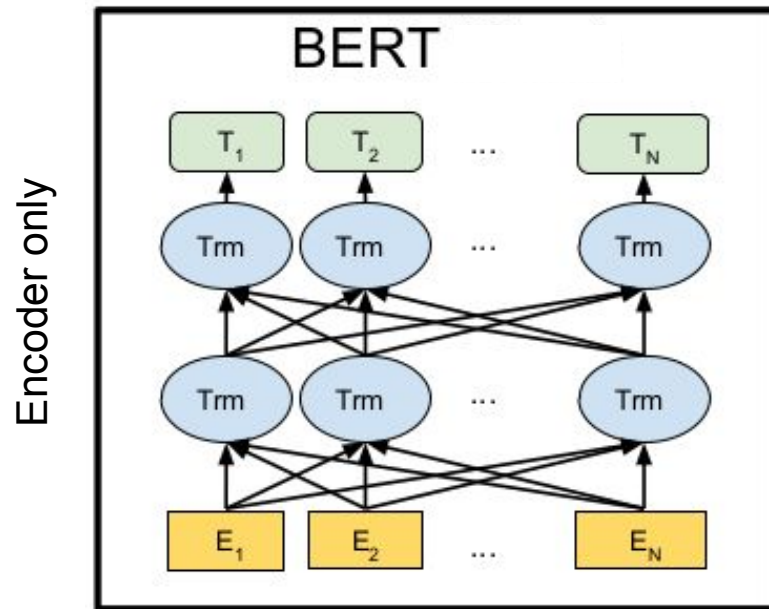


GPT vs BERT



Decoder only

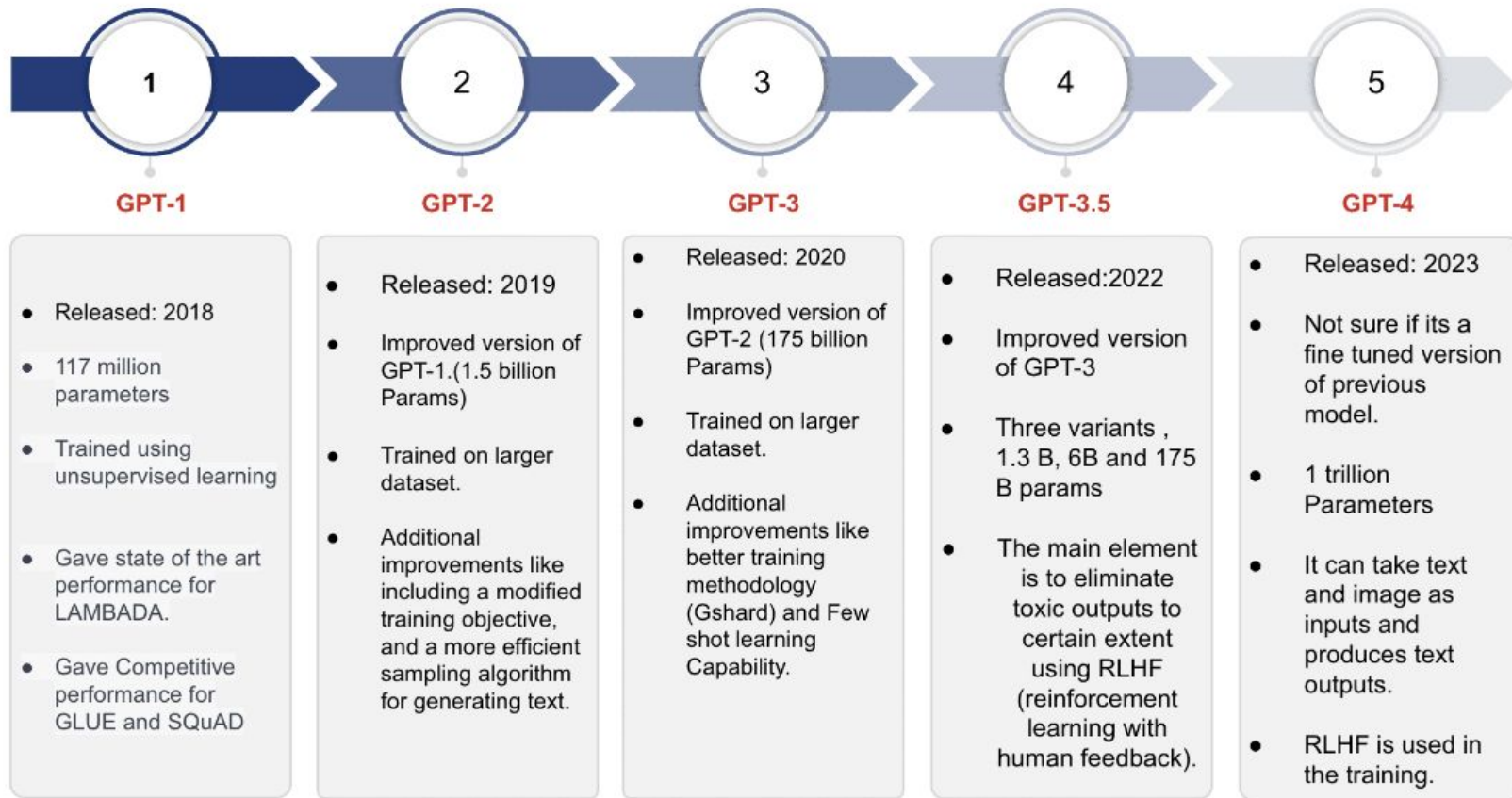
[Radford et al., 2018]



Encoder only

[Devlin et al., 2018]

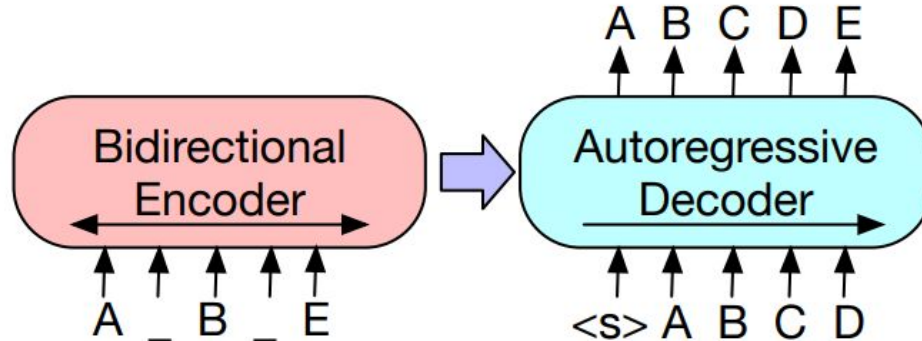
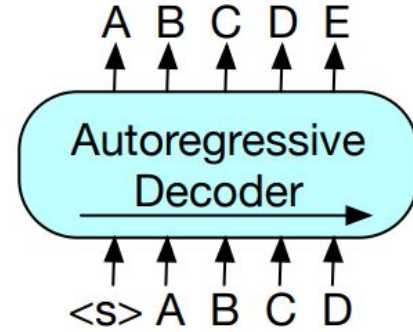
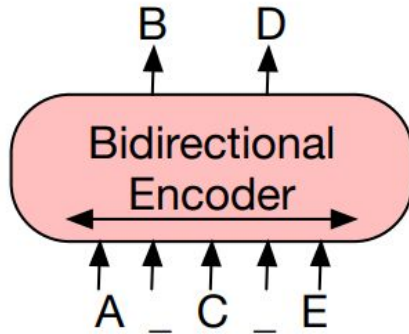
Evolution of GPTs



BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension

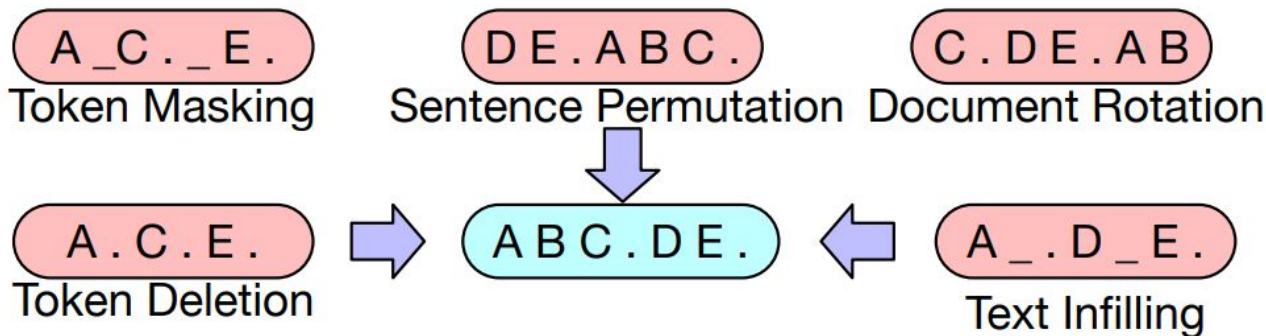
[Lewis et al., 2019]

BERT vs GPT vs BART



Denoising Operations

- **Token Masking:** Same as BERT.
- **Token Deletion:** Random tokens are deleted. Model predicts the positions of missing inputs.
- **Sentence Permutation:** Sentences are shuffled in a random order.
- **Document Rotation:** A randomly chosen token is made the first token. Model identifies the start of the document.
- **Text Infilling:** Multiple tokens (span) are masked with a single masked token.

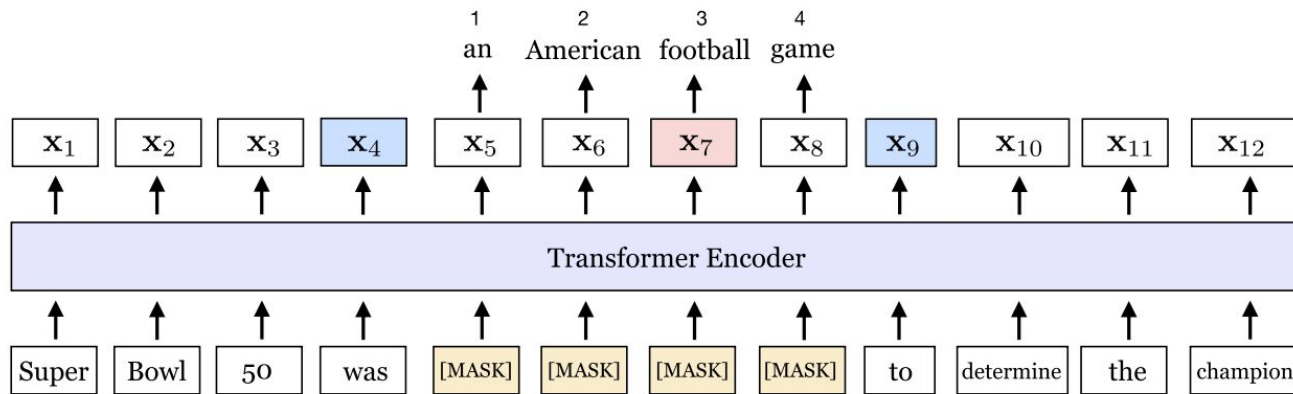


SpanBERT: Improving Pre-training by Representing and Predicting Spans

[Joshi et al., 2020] <https://aclanthology.org/2020.tacl-1.5.pdf>

SpanBERT

- Specifically designed for span prediction tasks (NER, Answer Extraction, etc.)
- Mask a sequence of tokens representing a span for prediction.



- Two objectives: MLM and SBO
 - Span-boundary objective (SBO) learns to predict the entire masked span from the observed tokens at its boundary.

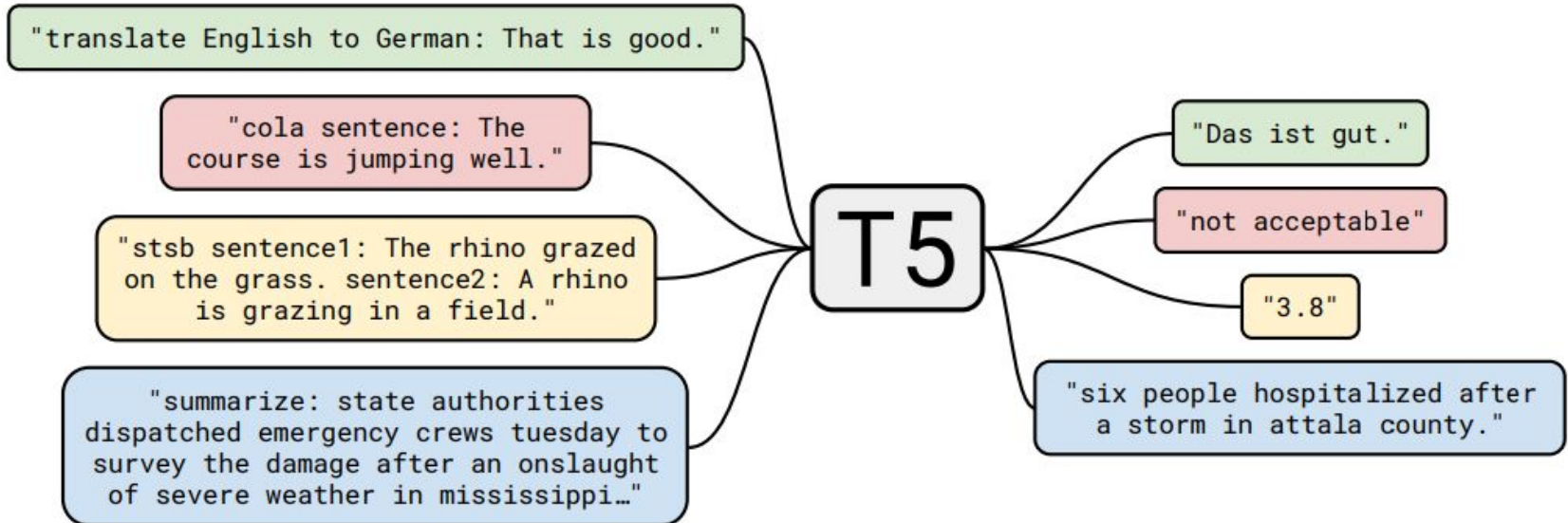
$$\begin{aligned}\mathcal{L}(\text{football}) &= \mathcal{L}_{\text{MLM}}(\text{football}) + \mathcal{L}_{\text{SBO}}(\text{football}) \\ &= -\log P(\text{football} \mid \mathbf{x}_7) - \log P(\text{football} \mid \mathbf{x}_4, \mathbf{x}_9, \mathbf{p}_3)\end{aligned}$$

T5: Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer

[Raffel et al., 2020] <https://arxiv.org/pdf/1910.10683.pdf>

Text-to-Text Transfer Transformer (T5)

- A unified model for generative and classification tasks.
- Its an encoder-decoder model pre-trained on a mixture of unsupervised and supervised tasks.
- Each task is converted into a text-to-text format.

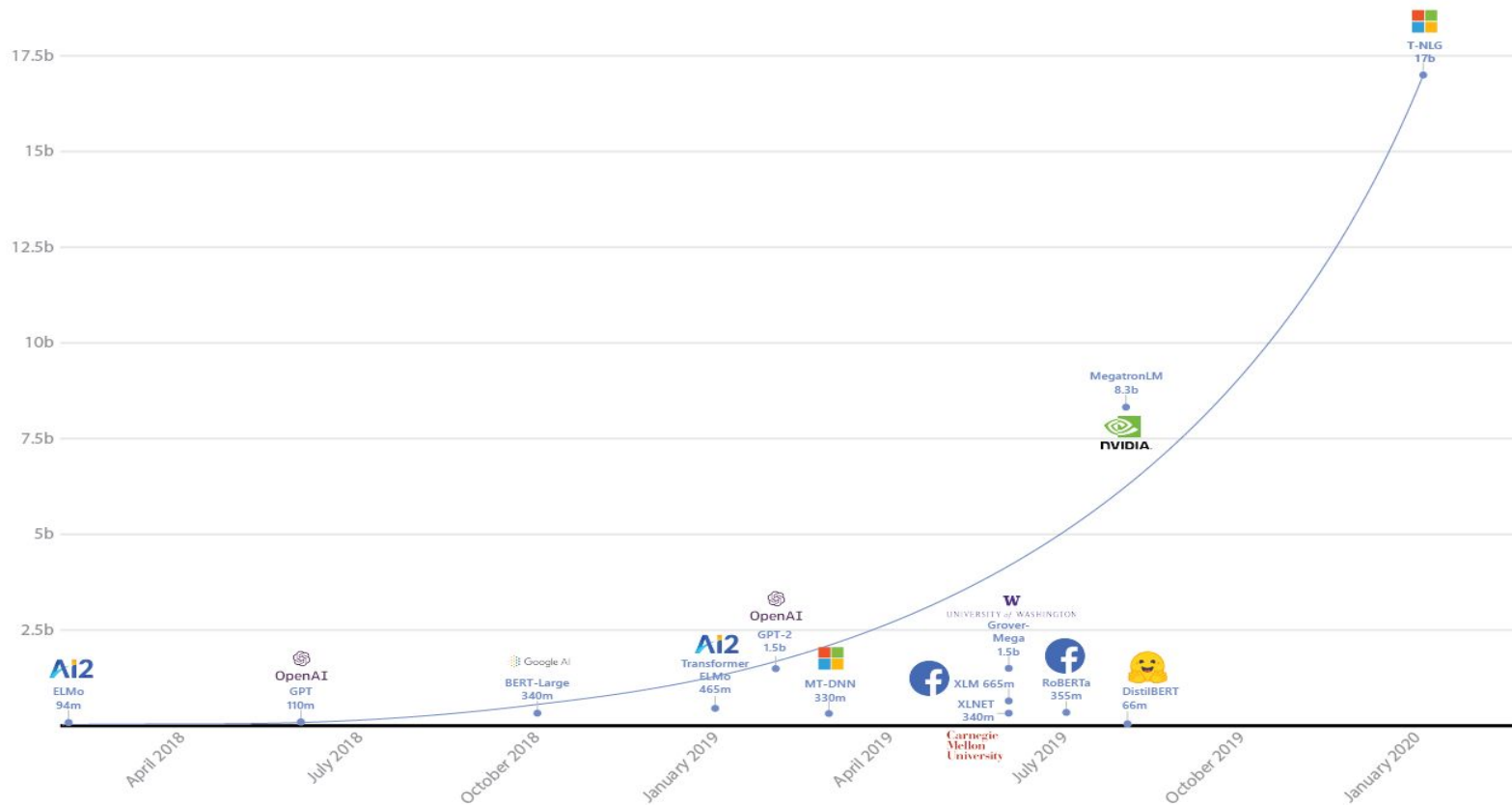


The LLM Race

GPT-4: 1.8T

Switch-C Transformers: 1.6T

OpenAI GPT-3: 175B





State-of-the-art Natural Language Processing for PyTorch and TensorFlow 2.0

🤖 Transformers provides thousands of pretrained models to perform tasks on texts such as classification, information extraction, question answering, summarization, translation, text generation, etc in 100+ languages. Its aim is to make cutting-edge NLP easier to use for everyone.

🤖 Transformers provides APIs to quickly download and use those pretrained models on a given text, fine-tune them on your own datasets then share them with the community on our [model hub](#). At the same time, each python module defining an architecture can be used as a standalone and modified to enable quick research experiments.

🤖 Transformers is backed by the two most popular deep learning libraries, [PyTorch](#) and [TensorFlow](#), with a seamless integration between them, allowing you to train your models with one then load it for inference with the other.

HuggingFace

- <https://github.com/huggingface/transformers>
- 250+ transformer-based models

1. **ALBERT** (from Google Research and the Toyota Technological Institute at Chicago)
2. **BART** (from Facebook)
3. **BARThez** (from École polytechnique)
4. **BERT** (from Google)
5. **BERT For Sequence Generation** (from Google)
6. **Blenderbot** (from Facebook)
7. **BlenderbotSmall** (from Facebook)
8. **BORT** (from Alexa)
9. **CamemBERT** (from Inria/Facebook/Sorbonne)
10. **ConvBERT** (from YituTech)
11. **CTRL** (from Salesforce)
12. **DeBERTa** (from Microsoft)
13. **DeBERTa-v2** (from Microsoft)
14. **DialoGPT** (from Microsoft Research)
15. **DistilBERT** (from HuggingFace),
16. **DPR** (from Facebook)
17. **ELECTRA**
18. **FlauBERT** (from CNRS)
19. **Funnel Transformer** (from CMU/Google Brain)
20. **GPT** (from OpenAI)
21. **GPT-2** (from OpenAI)
22. **I-BERT** (from Berkeley)
23. **LayoutLM** (from Microsoft Research Asia)
24. **LED** (from AllenAI)
25. **Longformer** (from AllenAI)

26. **LXMERT** (from UNC Chapel Hill)
27. **M2M100** (from Facebook)
28. **MarianMT**
29. **MBart** (from Facebook)
30. **MBart-50** (from Facebook)
31. **MPNet** (from Microsoft Research)
32. **MT5** (from Google AI)
33. **Pegasus** (from Google)
34. **ProphetNet** (from Microsoft Research)
35. **Reformer** (from Google Research)
36. **RoBERTa** (from Facebook),
37. **SpeechToTextTransformer** (from Facebook)
38. **SqueezeBert**
39. **T5** (from Google AI)
40. **TAPAS** (from Google AI)
41. **Transformer-XL** (from Google/CMU)
42. **Wav2Vec2** (from Facebook AI)
43. **XLNet** (from Facebook)
44. **XLNet-ProphetNet** (from Microsoft Research)
45. **XLNet-RoBERTa** (from Facebook AI)
46. **XLNet** (from Google/CMU)
47. **XLNet-Wav2Vec2** (from Facebook AI)